

3.1 Expression Statements

[This section corresponds to K&R Sec. 3.1]

Most of the statements in a C program are *expression statements*. An expression statement is simply an expression followed by a semicolon. The lines

```
i = 0;
i = i + 1;
and
printf("Hello, world!\n");
```

are all expression statements. (In some languages, such as Pascal, the semicolon separates statements, such that the last statement is not followed by a semicolon. In C, however, the semicolon is a statement terminator; all simple statements are followed by semicolons. The semicolon is also used for a few other things in C; we've already seen that it terminates declarations, too.)

Expression statements do all of the real work in a C program. Whenever you need to compute new values for variables, you'll typically use expression statements (and they'll typically contain assignment operators). Whenever you want your program to do something visible, in the real world, you'll typically call a function (as part of an expression statement). We've already seen the most basic example: calling the function `printf` to print text to the screen. But anything else you might do--read or write a disk file, talk to a modem or printer, draw pictures on the screen--will also involve function calls. (Furthermore, the functions you call to do these things are usually different depending on which operating system you're using. The C language does not define them, so we won't be talking about or using them much.)

Expressions and expression statements can be arbitrarily complicated. They don't have to consist of exactly one simple function call, or of one simple assignment to a variable. For one thing, many functions return values, and the values they return can then be used by other parts of the expression. For example, C provides a `sqrt` (square root) function, which we might use to compute the hypotenuse of a right triangle like this:

```
c = sqrt(a*a + b*b);
```

To be useful, an expression statement must do something; it must have some lasting effect on the state of the program. (Formally, a useful statement must have at least one *side effect*.) The first two sample expression statements in this section (above) assign new values to the variable `i`, and the third one calls `printf` to print something out, and these are good examples of statements that do something useful.

(To make the distinction clear, we may note that degenerate constructions such as

```
0;
i;
or
i + 1;
```

are syntactically valid statements--they consist of an expression followed by a semicolon--but in each case, they compute a value without doing anything with it, so the computed value is discarded, and the statement is useless. But if the "degenerate" statements in this paragraph don't make much sense to you, don't worry; it's because they, frankly, don't make much sense.)

It's also possible for a single expression to have multiple side effects, but it's easy for such an expression to be (a) confusing or (b) undefined. For now, we'll only be looking at expressions (and,

therefore, statements) which do one well-defined thing at a time.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)